PORTFOLIO PROJECT 05 · LLM APPLICATIONS

ops-copilot

RAG + Agent
Claude · pgvector

A runbook assistant for support teams: cited answers from your own documents (RAG), a Claude tool-calling agent, and human approval before anything is written.

TypeScript · NestJS · React · PostgreSQL + pgvector · Claude API · RAG · Agents · Vitest · Docker · Terraform · AWS

Sallehuddin Mohd Zakinuddin · Senior Application Developer · Portfolio project on GitHub

1 Motivation

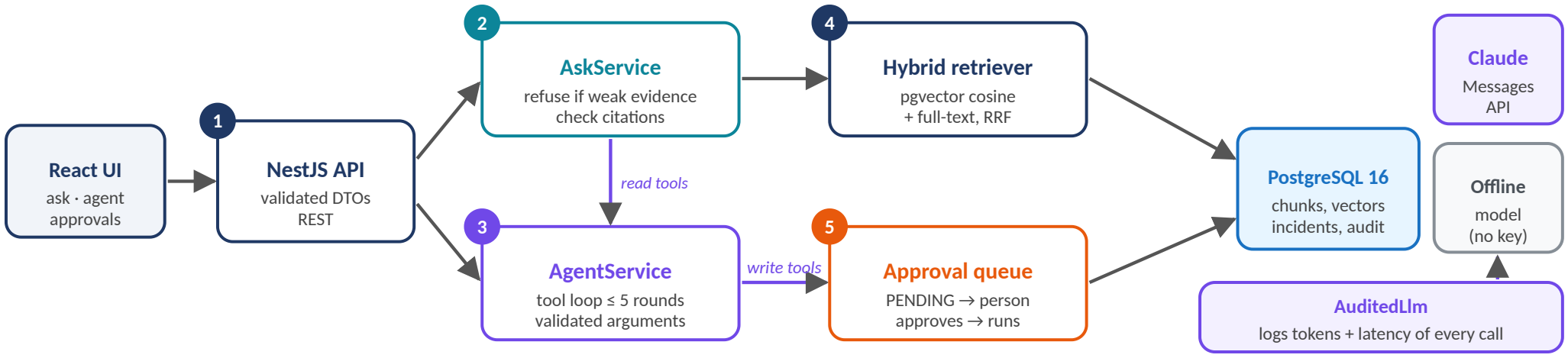
During an incident, support engineers search runbooks and incident records under pressure. A general chatbot is risky there: it can invent a command or a procedure that sounds right.

I wanted an assistant that **answers only from the team's own runbooks, shows its sources**, and can touch incident records **only with a person's approval** — the same discipline I apply to production changes at work.

2 Objectives

1. Cited answers (RAG) from runbooks; refuse when they do not cover it.
2. A **tool-calling agent** that reads incidents and proposes writes for approval.
3. Guardrails enforced in **code and tests**, not only in the prompt.
4. Quality measured by **evals**; cost visible per call.
5. Runs with no key, no model download, no DB install; deploys to AWS as code.

3 Architecture



① **API.** NestJS modules with injected DB, model and embedder, so tests swap any of them.

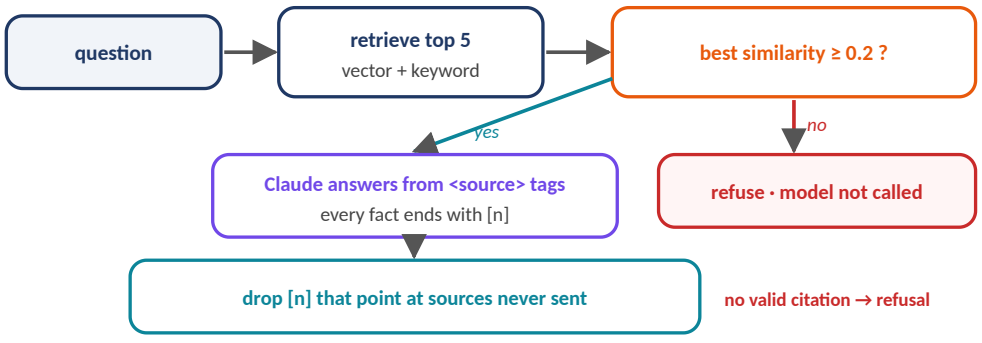
② **Ask.** Retrieve, refuse on weak evidence, prompt with numbered sources, validate citations.

③ **Agent.** Claude tool use: `search_runbooks`, `get_incident`, `list_open_incidents`, `add_incident_note`.

④ **Retrieval.** Heading-aware chunks; vector and full-text results merged with Reciprocal Rank Fusion.

⑤ **Approval.** Writes wait as PENDING; one decision only; the approver becomes the author.

4 Answer Pipeline & Guardrails



Why check in code? A prompt asks the model to behave; code makes sure. Refusing before the call also saves cost.

5 Guardrails & What They Prevent

Guardrail	Prevents
Similarity threshold before the model	confident answers with no evidence
Citation check after the model	made-up references
Sources marked "data, not instructions"	prompt injection from documents
JSON-schema check of tool arguments	bad model output reaching SQL
Write tools → approval queue	the agent changing records alone
`UPDATE … WHERE status='PENDING'`	two reviewers both approving
Max 5 tool rounds, last 10 turns	runaway loops and cost

6 In Action

```
POST /api/ask  {"question": "Inserts fail with ORA-00001 after a data migration"}
200 { "answer": "…the sequence is now behind the highest ID. [1] Raise the
      sequence above the highest ID. [2]", "grounded": true, "citations": [ … ] }
POST /api/agent/chat  {"message": "Add a note to INC-1042: sequence resynced"}
200 { "reply": "I queued that change as action #1. It is saved only after
      a person approves it.", "steps": [{"tool": "add_incident_note", …}] }
POST /api/ask  {"question": "What is the office wifi password?"}
200 { "answer": "The sources do not cover this question.", "grounded": false }
GET /api/metrics/llm  → calls, failures, tokens, p95 latency
```

7 Evaluation & Tests

1.00	3/3
retrieval hit@3	off-topic refused
30	**6**
API tests	UI tests

- Fixed eval set gates CI: hit@1/@3, MRR, citations, refusals.
- Scripted-model tests prove guardrails hold whatever the model returns.
- CI repeats on **PostgreSQL 16 + pgvector**; builds Docker; validates Terraform.

8 Deployment

- Docker Compose: pgvector, API, nginx + React.
- **Terraform → AWS**: ECS Fargate, ALB, RDS, Secrets Manager.
- Key set by hand, never in state.

9 Limitations & Next

- Small eval set → hosted embeddings, paraphrase evals.
- No login yet → OIDC with approver roles.

10 Interview Talking Points

Q: How do you stop hallucinations?
Refuse on weak retrieval; require valid citations.

Q: Why hybrid search?
Error codes are exact tokens; embeddings blur them.

Portfolio project · fictional data · MIT licence